

# MEMORIAL TÉCNICO DE DESENVOLVIMENTO

**Projeto:** Portal de Solicitações Internas — 2ª etapa do processo seletivo, Desenvolvedor(a) de Sistemas Júnior (bit Soluções)

**Candidato:** Samuel Andrade de Araújo

**Data de entrega:** 05/10/2026

## 1. Visão geral

O Portal de Solicitações Internas permite que colaboradores registrem demandas (TI, RH, Compras, Financeiro, Infraestrutura) e acompanhem sua evolução até a conclusão. A solução é composta por três partes independentes, orquestradas por Docker Compose:

Navegador → nginx (Angular SPA, :8081) —/api→ Spring Boot (:8080) → PostgreSQL (:5432)

O objetivo deste memorial é registrar **o que foi usado, por que foi escolhido e quais decisões foram tomadas** (inclusive onde o enunciado era ambíguo), além de uma análise crítica das limitações da solução.

## 2. Tecnologias utilizadas

### Backend

| Tecnologia                  | Versão                 | Uso                                        |
|-----------------------------|------------------------|--------------------------------------------|
| Java                        | 25 (LTS)               | Linguagem                                  |
| Spring Boot                 | 4.1.1                  | Framework da API (Web MVC, Actuator)       |
| Spring Data JPA / Hibernate | (gerenciado pelo Boot) | Persistência e consultas                   |
| Spring Security             | (gerenciado pelo Boot) | Autenticação por sessão, CSRF, BCrypt      |
| Bean Validation (Jakarta)   | (gerenciado pelo Boot) | Validação declarativa de entrada           |
| Flyway                      | (gerenciado pelo Boot) | Versionamento do esquema do banco          |
| springdoc-openapi           | 3.1.1                  | Documentação OpenAPI / Swagger UI          |
| Lombok                      | (gerenciado pelo Boot) | Redução de código repetitivo nas entidades |
| Maven                       | 3.9 (wrapper)          | Build                                      |

## Frontend

| Tecnologia             | Versão       | Uso                                                                                  |
|------------------------|--------------|--------------------------------------------------------------------------------------|
| TypeScript             | 6.0          | Linguagem                                                                            |
| Angular                | 22.2         | Framework da SPA (componentes standalone, signals, rotas lazy, formulários reativos) |
| Angular Material / CDK | 22.2         | Componentes de interface (tabela, formulários, diálogo, paginador, datepicker)       |
| RxJS / HttpClient      | (do Angular) | Comunicação com a API                                                                |
| Vitest                 | 5            | Testes unitários e de componentes                                                    |
| Node.js / npm          | 24 / 11      | Ferramentas de build                                                                 |

## Dados, infraestrutura e qualidade

| Tecnologia                             | Versão | Uso                                                     |
|----------------------------------------|--------|---------------------------------------------------------|
| PostgreSQL                             | 16     | Banco de dados relacional                               |
| Docker / Docker Compose                | —      | Empacotamento e execução em um comando                  |
| nginx                                  | 1.29   | Serve a SPA e faz proxy de /api                         |
| JUnit 5, MockMvc, Spring Security Test | —      | Testes do backend                                       |
| Testcontainers                         | —      | PostgreSQL real nos testes de integração                |
| GitHub Actions                         | —      | Integração contínua (build e testes do back e do front) |

## 3. Justificativa técnica

Para cada tecnologia: **motivo**, **benefício**, **vantagem sobre alternativas** e **impacto em manutenção/escalabilidade**.

### 3.1 Java 25 e Spring Boot 4

- **Motivo:** o enunciado menciona aderência às tecnologias do perfil da vaga; Spring Boot é o padrão de mercado para APIs corporativas em Java.
- **Benefícios:** configuração automática, ecossistema integrado (segurança, dados, validação, testes) e convenções que reduzem decisões repetitivas.
- **Vs. alternativas:** frente a montar uma API com servlets/bibliotecas isoladas, entrega muito mais com menos código; frente a frameworks mais enxutos (Javalin, Micronaut), tem maior base de conhecimento e documentação. Java 25 é uma versão LTS, com suporte de longo prazo.
- **Impacto:** a estrutura em camadas e a injeção de dependências facilitam testes e evolução; a aplicação é *stateless* exceto pela sessão (ver 5.5), o que permite escalar horizontalmente com sessão compartilhada.

### 3.2 Spring Data JPA com `Specification`

- **Motivo:** o acesso a dados é majoritariamente CRUD, e a listagem exige filtros opcionais combináveis (período, categoria, status e texto).
- **Benefícios:** repositórios declarativos e consulta dinâmica tipada ( `Specification` ) sem concatenar SQL; `@EntityGraph` carrega categoria e solicitante na mesma consulta, evitando N+1.
- **Vs. alternativas:** JDBC puro exigiria montar SQL manualmente (risco de injeção e mais código); MyBatis/jOOQ são ótimos, mas seriam mais cerimônia para um CRUD com filtros simples.
- **Impacto:** o mapeamento fica concentrado no domínio; mudar de banco é pouco invasivo. O custo é a curva de aprendizado do JPA (lazy loading); mitigado com `open-in-view=false` e mapeamento para DTOs dentro da transação.

### 3.3 PostgreSQL

- **Motivo:** o enunciado exige banco SQL; modelo relacional encaixa naturalmente (usuários, categorias, solicitações com chaves estrangeiras).
- **Benefícios:** integridade referencial, `CHECK`, `TIMESTAMPTZ` (datas sem ambiguidade de fuso), bons índices e licença livre.
- **Vs. alternativas:** SQLite seria mais simples, mas não representa um ambiente multiusuário; MySQL atenderia, porém o PostgreSQL tem tipos e constraints mais ricos. NoSQL não faz sentido para dados fortemente relacionais.
- **Impacto:** escala bem verticalmente e oferece replicação para leitura quando necessário.

### 3.4 Flyway

- **Motivo:** o enunciado pede mecanismo para criar a estrutura do banco (scripts SQL ou equivalente).
- **Benefícios:** scripts SQL versionados ( `V1__...`, `V2__...` ), aplicados automaticamente e em ordem na subida; o Hibernate apenas **valida** o esquema ( `ddl-auto=validate` ), então o banco nunca diverge do que está no repositório.
- **Vs. alternativas:** `ddl-auto=update` é conveniente, porém não versiona e pode causar alterações imprevisíveis; Liquibase é equivalente, mas com XML/YAML em vez de SQL puro, menos direto para este caso.
- **Impacto:** evolução do esquema rastreável e reproduzível entre ambientes.

### 3.5 Spring Security (sessão + CSRF)

- **Motivo:** autenticação e controle de sessão são requisitos explícitos.
- **Benefícios:** senhas com BCrypt, proteção CSRF, troca do identificador de sessão no login (contra *session fixation*), cookie `HttpOnly` e `SameSite=Lax`, e resposta 401 padronizada para rotas protegidas.
- **Vs. alternativas:** JWT em `localStorage` expõe o token a XSS e exige reinventar revogação/expiração; para uma SPA servida na mesma origem da API, sessão por cookie é mais simples e mais segura. Implementar autenticação manualmente seria reinventar a roda com maior risco.
- **Impacto:** o controle fica centralizado em uma configuração; para escalar horizontalmente seria necessário um repositório de sessão compartilhado (Spring Session + Redis/JDBC).

### 3.6 Bean Validation e tratamento global de erros

- **Motivo:** o enunciado pede "tratamento adequado de erros e validações".
- **Benefícios:** regras declaradas junto aos DTOs ( `@NotBlank` , `@Size` ) e um `@RestControllerAdvice` que converte toda exceção em `ProblemDetail` (RFC 9457), com mapa `erros` por campo nas validações. O cliente recebe sempre o mesmo formato, e detalhes internos (stack trace) nunca vazam.
- **Vs. alternativas:** `if` s manuais espalhados em controllers geram inconsistência.
- **Impacto:** o frontend consome um único formato de erro; novos endpoints herdam o comportamento.

### 3.7 springdoc-openapi

- **Motivo:** facilitar a avaliação e o consumo da API.
- **Benefício:** documentação interativa gerada do código, sempre sincronizada.
- **Vs. alternativas:** documentar à mão em Markdown desatualiza facilmente.
- **Impacto:** custo de manutenção quase nulo. Em produção seria desabilitado ou protegido (ver 7).

### 3.8 Angular 22 (standalone components, signals)

- **Motivo:** o perfil da vaga cita frameworks de frontend e o Angular entrega estrutura opinativa (rotas, formulários, HTTP, DI) sem precisar escolher bibliotecas avulsas.
- **Benefícios:** componentes standalone com *lazy loading* por rota, *signals* para estado local simples, formulários reativos com validação, `HttpClient` com suporte nativo a XSRF, tipagem forte de ponta a ponta com TypeScript.
- **Vs. alternativas:** React/Vue são mais flexíveis, mas exigem decidir roteamento, formulários e estrutura por conta própria; num time, a convenção do Angular reduz divergência.
- **Impacto:** a estrutura por *features* ( `core` , `layout` , `features` , `shared` ) permite crescer sem acoplamento; *lazy loading* mantém o carregamento inicial enxuto.

### 3.9 Angular Material

- **Motivo:** entregar uma interface utilizável e acessível dentro do prazo de cinco dias.
- **Benefícios:** componentes prontos e testados (tabela, paginador, datepicker, diálogo), tema consistente, acessibilidade (ARIA, foco) e responsividade.
- **Vs. alternativas:** CSS próprio custaria muito tempo; Bootstrap/Tailwind exigiriam montar componentes complexos (datepicker, select) manualmente.
- **Impacto:** padronização visual e manutenção simples; o custo é o tamanho do bundle (ver 7).

### 3.10 Testes: JUnit, MockMvc, Testcontainers e Vitest

- **Motivo:** demonstrar que as regras de negócio funcionam e permitir refatorar com segurança.
- **Benefícios:** os testes de integração rodam contra **PostgreSQL real** (Testcontainers) em vez de um banco em memória, então validam migrations, constraints e consultas exatamente como em produção. No frontend, Vitest roda rápido e integra-se ao `ng test` .
- **Vs. alternativas:** H2 poderia mascarar diferenças de dialeto SQL.
- **Impacto:** suíte reproduzível na CI; é o que sustenta a manutenção do projeto.

### 3.11 Docker, Docker Compose e nginx

- **Motivo:** o avaliador precisa executar o sistema sem instalar Java, Node ou PostgreSQL.
- **Benefícios:** `docker compose up --build` sobe banco, API e site; builds *multi-stage* mantêm as imagens de execução pequenas (site: 95 MB) e o processo da API roda como usuário não-root; *healthchecks* garantem a ordem de subida.
- **Vs. alternativas:** instruções manuais de instalação tendem a falhar em máquinas diferentes.
- **Impacto:** ambiente idêntico entre desenvolvimento, CI e avaliação. O nginx serve a SPA e encaminha `/api`, de modo que front e API compartilham a **mesma origem** (sem CORS e com cookies de sessão/CSRF funcionando).

### 3.12 GitHub Actions

- **Motivo:** impedir regressões a cada push/PR.
- **Benefício:** executa build e testes de backend e frontend automaticamente.
- **Impacto:** feedback rápido; é a base para um futuro pipeline de entrega.

---

## 4. Modelagem de dados

Modelo relacional com três tabelas (detalhes completos em [dicionario-de-dados.md](#)):

```
usuarios 1 —< solicitacoes >— 1 categorias
```

Decisões:

- **Status como texto + CHECK** em vez de tabela de domínio: são três valores fixos, atrelados à regra de negócio (fluxo sequencial). Ficam no enum Java `StatusSolicitacao` e protegidos por constraint no banco.
- **Categorias em tabela:** a lista pode evoluir sem alterar código.
- **Código ( SOL-000123 ) derivado do id**, e não uma coluna: evita duplicidade e dessincronia.
- **TIMESTAMP TZ** em todas as datas, evitando ambiguidade de fuso nos filtros por período.
- **Controle de concorrência otimista** (`@Version`, coluna `versao`): duas gravações simultâneas na mesma solicitação (por exemplo, a edição de um usuário e o avanço de status de outro) não se sobrescrevem; a segunda recebe HTTP 409 com orientação para atualizar a página. Sem isso, o `UPDATE` de colunas completas do Hibernate poderia reverter um status recém-alterado.
- **Constraints no banco** (não nulos, `CHECK`, FKs, unicidade) como última linha de defesa, além das validações da aplicação.
- **Índices** em status, categoria, solicitante e data de criação, que são os critérios de filtro.

## 5. Justificativa conceitual (arquitetura)

---

### 5.1 Estrutura geral

SPA (Angular) + API REST (Spring Boot) + banco relacional, três componentes com responsabilidades claras e comunicação apenas via HTTP/JSON. O frontend não conhece o banco; a API não conhece a interface.

### 5.2 Organização das camadas (backend)

`controller` → `service` → `repository` → banco, com DTOs na borda:

- **controller**: traduz HTTP em chamadas ao serviço; não contém regra de negócio.
- **service**: concentra regras de negócio, permissões e transações ( `SolicitacaoService` , `DashboardService` ).
- **repository**: acesso a dados (Spring Data); consultas dinâmicas em `SolicitacaoSpecs` .
- **domain**: entidades e o enum de status, incluindo a lógica do fluxo ( `proximo()` , `podeIrPara()` ).
- **dto**: *records* imutáveis de entrada/saída; entidades JPA nunca são expostas pela API, o que também impede *mass assignment* (campos como `status` ou `solicitanteId` enviados no corpo são ignorados).
- **exception**: exceções de domínio e tratamento global.

### 5.3 Regras de negócio na API, não na interface

Toda regra é imposta no servidor; a interface apenas a reflete:

- solicitante e data de criação vêm da sessão e do relógio do servidor;
- editar/excluir: 404 se não existe → 403 se não é o dono → 409 se não está Aberta;
- status: só avança ao próximo passo (409 caso contrário).

A resposta já carrega `editavel` e `proximoStatus` , calculados pelo servidor, para o frontend não duplicar regras (uma única fonte da verdade).

### 5.4 Padrões de projeto

Injeção de dependências (construtor), *Repository*, *Specification* (consulta dinâmica), *DTO*, *Service Layer*, *Controller Advice* (tratamento centralizado de erros), *Guard* e *Interceptor* no Angular, e *Smart/Dumb components* na medida em que o projeto exigiu (ex.: `StatusChip` , `ConfirmDialog` reutilizáveis).

### 5.5 Estratégia de autenticação

Sessão por cookie ( `JSESSIONID` , `HttpOnly` , `SameSite=Lax` , expiração de 30 minutos de inatividade) com proteção CSRF por *double-submit cookie*: a API publica `XSRF-TOKEN` e o Angular devolve o valor em `X-XSRF-TOKEN` nas requisições de escrita (o `HttpClient` faz isso automaticamente). No Angular, um `authGuard` protege as rotas e um *interceptor* trata respostas 401 (sessão expirada), levando o usuário ao login e retornando-o depois à página original (apenas destinos internos são aceitos). Ao iniciar, o app consulta `/api/auth/me` , de modo que recarregar a página não derruba a sessão.

## 5.6 Comunicação entre frontend e backend

JSON sobre HTTP, na mesma origem via proxy do nginx (e proxy do `ng serve` em desenvolvimento). Contrato com datas em ISO-8601, paginação ( `pagina` , `tamanho` ) e erros em `application/problem+json` . A documentação do contrato é gerada pelo springdoc.

## 5.7 Organização do código-fonte

Monorepo com `backend/` , `frontend/` e `docs/` . No frontend, organização por *features* ( `login` , `dashboard` , `solicitacoes` ), com `core` (serviços singletons, guards, interceptor), `layout` e `shared` . Nomenclatura em português para o domínio (que é o vocabulário do enunciado) e em inglês apenas para termos técnicos do framework.

## 5.8 Segurança básica aplicada

BCrypt para senhas; CSRF; sessão `HttpOnly` ; troca de ID de sessão no login; consultas parametrizadas (JPA); escape de `% / _` na busca por título; entidades não expostas; mensagens de erro sem detalhes internos; usuário não-root no container; redirecionamento pós-login restrito a rotas internas; limite de tamanho de página (50).

---

## 6. Decisões sobre requisitos ambíguos

---

O enunciado deixa alguns pontos em aberto. Decisões tomadas:

| Ponto                               | Decisão                                                                                                              | Justificativa                                                               |
|-------------------------------------|----------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| Transição de status                 | <b>Sequencial</b> (Aberto → Em Atendimento → Concluído), sem pular nem retroceder                                    | Reflete um fluxo de atendimento real e evita estados incoerentes            |
| Perfis de usuário                   | <b>Perfil único</b> : todos veem todas as solicitações; só o dono edita/exclui; qualquer autenticado avança o status | O enunciado não define atendentes/administradores; mantém o escopo simples  |
| "Editar/excluir solicitação aberta" | Restrito ao <b>solicitante</b> e a solicitações <b>Abertas</b> , validado na API                                     | Impede alteração por terceiros e preserva o histórico de itens em andamento |
| Período de busca                    | Datas inclusivas (início e fim), interpretadas no fuso <code>America/Sao_Paulo</code>                                | Evita que um registro "de hoje" caia fora do filtro por diferença de fuso   |
| Cadastro de usuários                | Sem tela de cadastro; dois usuários de demonstração criados por migration                                            | Não faz parte dos requisitos; documentado como ressalva                     |
| Exclusão                            | Física (remove o registro)                                                                                           | O enunciado não pede histórico; ver melhorias futuras                       |

---



## 7. Análise crítica

---

### Limitações da solução atual

- **Perfis:** não há papel de atendente/administrador; qualquer usuário autenticado pode avançar o status de qualquer solicitação.
- **Usuários:** sem cadastro, recuperação de senha, bloqueio por tentativas falhas nem política de senha. Os usuários de demonstração e suas senhas fixas estão em uma migration, apenas para facilitar a avaliação.
- **Sessão em memória:** a sessão fica na memória da instância da API; com várias instâncias seria necessário compartilhá-la.
- **Sem histórico/auditoria:** a mudança de status não registra quem alterou nem quando; a exclusão é física.
- **Sem anexos, comentários ou notificações**, comuns em portais de solicitação.
- **Listagem:** ordenação fixa (mais recentes primeiro); o usuário não escolhe a coluna de ordenação.
- **Frontend:** o *bundle* inicial tem ~650 kB (153 kB comprimido), acima do limite padrão do Angular (500 kB), e foi mantido com o orçamento ajustado para 700 kB; fontes e ícones vêm do Google Fonts (CDN), exigindo internet no navegador.
- **Testes:** não há testes de ponta a ponta no navegador (Playwright/Cypress); a verificação da interface foi manual e por testes de componente.

### Melhorias futuras

- Perfis (solicitante, atendente, administrador) com `@PreAuthorize`, atribuição de responsável e tela de administração de usuários.
- Tabela de histórico ( `solicitacao_historico` ) e exclusão lógica (*soft delete*) para auditoria.
- Comentários e anexos; notificações por e-mail; SLA e prazos.
- Ordenação e busca avançada na listagem; exportação (CSV/PDF).
- Testes E2E com Playwright; análise estática e cobertura na CI.
- Imagem Docker da API menor (jlink/jar em camadas); fontes e ícones empacotados localmente.
- Internacionalização formal ( `@angular/localize` ) caso o sistema precise de mais de um idioma.

### Requisitos que poderiam ser aperfeiçoados no enunciado

- Definir **perfis e permissões** (quem pode alterar status? quem pode ver o quê?).
- Definir o **fluxo de status** (sequencial ou livre; é possível reabrir?).
- Dizer se a exclusão deve ser **física ou lógica**.
- Esclarecer se a pesquisa por período considera a data de abertura ou a de atualização.

### O que seria diferente em um ambiente corporativo de produção

- **Autenticação federada** (SSO/OIDC, por exemplo Keycloak ou Entra ID) em vez de usuários e senhas locais; sessão compartilhada ou *tokens* de curta duração.
- **HTTPS obrigatório** (cookie `Secure`, HSTS) atrás de um *reverse proxy*/balanceador; CSRF e CORS revistos para a topologia real.
- **Segredos** em um cofre (não em variáveis de ambiente em texto, e senhas do banco diferentes das de demonstração); banco **não exposto** ao host e Swagger desabilitado ou protegido.



- **Observabilidade:** logs estruturados, métricas (Micrometer/Prometheus), *tracing* e alertas; *healthchecks* de liveness/readiness.
- **Banco gerenciado** com *backups*, política de retenção e migrations aplicadas por etapa de pipeline, com *rollback* planejado (e `ddl-auto=validate` mantido).
- **Pipeline de entrega** completo (build, testes, varredura de vulnerabilidades, publicação de imagens versionadas e *deploy* automatizado), com ambientes separados.
- **Limitação de taxa** (*rate limiting*) no login e proteção contra *brute force*.
- **Política de dados:** LGPD (retenção, exclusão de dados pessoais) e trilha de auditoria.

## 8. Testes e verificação

| Suíte                                     | Quantidade | O que cobre                                                                                                                                                                                                                                    |
|-------------------------------------------|------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Backend<br>(JUnit/MockMvc/Testcontainers) | 37         | Autenticação, CSRF, CRUD, permissões (403/409/404), fluxo de status, filtros (período inclusivo, status, categoria, título com escape de curingas), paginação, dashboard, validações (400), concorrência otimista, migrations e mapeamento JPA |
| Frontend (Vitest)                         | 29         | Serviço de autenticação (inclui a reemissão do token CSRF no logout), guards, interceptor de 401, serviço de solicitações, utilitários, componentes (login, listagem, detalhe, chip de status)                                                 |

Além dos testes automatizados, a aplicação completa foi executada via `docker compose up --build` e percorrida manualmente (login, criação, edição, avanço de status, filtros, exclusão, logout seguido de novo login, versão mobile). Uma revisão de código posterior encontrou e corrigiu três problemas: o logout apagava o token CSRF e fazia o login seguinte falhar, uma condição de corrida entre edição e mudança de status (corrigida com `@Version`) e filtros da listagem que não acompanhavam a URL. As capturas estão em [evidencias/](#).